Circuit Simulation Code Overview

Data Structure Setup

The code assumes a specific format for the input file, where lines of the circuit description specify:

- **Resistors (R)**
- **Voltage Sources (V)**
- **Current Sources (I)**

The connections between these elements are also specified in the input.

Functions

The code includes various functions to handle distinct tasks:

- `get_nodes`: Extracts and returns a list of distinct nodes in the circuit.
- `get_vol_sources`: Counts and returns the number of voltage sources.
- `get_curr_sources`: Counts and returns the number of current sources.
- `Convert_vol_sources`: Standardizes the naming of voltage sources (e.g., converting "Vsource" to "V1").
- `Convert_cur_sources`: Standardizes the naming of current sources similarly.
- `Convert_nodes`: Converts node names to a standardized numeric format.
- `G_matrix`, `B_matrix`, `C_matrix`, **and** `D_matrix`: These functions construct matrices used to formulate the circuit equations based on Kirchhoff's laws.
- `z`: Constructs the z matrix, which contains the current sources and voltage sources.
- `get_voltages` **and** `get_currents`: After solving the linear equations, these functions extract voltage and current values from the result.
- `check_invalid_element`: Validates the input, raising an error if any unexpected elements are present in the circuit description.
- `evalSpice`: The main function that orchestrates reading the circuit file, handling the data, constructing matrices, solving the equations, and returning the results.

The Circuit Analysis Process

File Reading and Parsing

The circuit description is read from a file. The `.circuit` and `.end` keywords are used to isolate the relevant lines that constitute the circuit.

Matrix Construction

Several matrices (`G`, `B`, `C`, `D`) are constructed based on the circuit's components and their interconnections:

- **G Matrix**: Represents the conductance relationships in the circuit derived from the resistors.
- **B Matrix**: Represents the contributions of the voltage sources to the circuit equations.
- **C Matrix**: Transpose of the B matrix, capturing the relationship for voltage sources.
- **D Matrix**: A matrix of zeros that has dimensions based on the number of voltage sources.

Linear Algebra Solution

The system of equations (`Ax = z`) (where `A` combines `G`, `B`, `C`, and `D` matrices, and `z` incorporates current sources and voltage levels) is solved using NumPy's linear algebra functionality.

Output Generation

After solving, voltages at the nodes and currents through the voltage sources are extracted and returned.

Conclusion

The code provides a systematic approach to simulating circuits using node-voltage analysis. By breaking the setup into logical functions and carefully constructing the matrices, the code can effectively handle small circuits defined in a text format, making it useful for educational purposes or simple circuit simulations. Its modular design allows for easy modifications or extensions, such as adding new circuit elements or enhancing the parsing logic for different file formats.

Resources used

Resource

Test cases covered

- `Invalid element error`: If any element except R,V or I occurs then it raises a value error.
- `Malformed circuit error`: If the circiut description doesnt start with .circiut or end with .end, it raises a value error.
- `Only current sources error`: If the circuit has no element except current sources in series or parallel , then it is an impossible matrix and it raises a value error.
- `Only voltage sources error`: If the circuit has no element except voltage sources in series or parallel , then it is an impossible matrix and it raises a value error.
- `Wrong node name error`: If while getting a node voltage or current, if someone enters a wrong name then it raises a key error.
- `File not found error`: If no file given then raise the given error